

Politecnico di Torino
Department of Electronics and Telecommunications
Master's degree in Mechatronic Engineering



Sensors, Embedded Systems and Algorithms for Service Robotics

2025/2026

Report 2¹

Date, 14/12/2025

Group n° 14

Luigi Muratore — S333098

Francesco Licci — S338759

Davide Tocci — S344917

Peter Furlan — S344945

Marat Perez — S336992

¹GitHub Repository: <https://github.com/luigimuratore/SES4SR-ros>

Objective

Developing and validating a ROS 2 package for robot localization that implements an Extended Kalman Filter (EKF), fusing sensor data (odometry, IMU, and visual landmarks) to estimate the robot's pose more accurately than standard dead-reckoning methods.

Goals

In particular, the goals were:

- Implement probabilistic velocity-based motion models and landmark measurement models in Python, computing their Jacobian matrices for the EKF linearization.
- Design a ROS 2 node that predicts the robot state using velocity commands and corrects it using range-and-bearing measurements from detected AprilTag landmarks.
- Extend the filter state to include linear and angular velocities, fusing data from the IMU and wheel encoders to improve the robustness of the localization.
- Record data from all relevant topics using ROSbag and evaluate the filter performance by computing RMSE and MAE metrics against ground truth.
- Deploy the localization package on the physical TurtleBot3 robot.

Design

Task 0: Probabilistic Model Verification

This task focuses on implementing and validating the probabilistic models, in order to test and verify uncertainty propagation before integrating the models into the EKF.

Task 0a: Probabilistic Velocity Motion Model

This task focuses on implementing and verifying the prediction step of the filter using a velocity-based motion model.

Symbolic Jacobian Computation The script uses the sympy library to define the motion model symbolically:

- State & Control: variables are defined as symbols $(x, y, \theta, v, \omega, dt)$.
- Motion Function: the function $g(u, x)$ is constructed as a SymPy Matrix, representing the kinematic evolution of the robot
- Instead of manually deriving derivatives, the script calls `.jacobian()` to automatically compute G_t (jacobian w.r.t. state) and V_t (w.r.t. control).
- The symbolic matrices are converted into fast, callable Numpy functions using `sympy.lambdify` for efficient numerical evaluation during runtime.

Sampling and visualization The script generates 500 samples ($n_samples = 500$) to approximate the posterior distribution of the robot's pose.

It then visualizes the resulting particle cloud using matplotlib. A key design detail is the use of `rotated_marker._transform`, which rotates the arrow markers to visually represents the orientation (θ) of each sample, not just its (x, y) position.

Noise parameters analysis The noise vector α dictates how uncertainty propagates through the system. The code implements a list of six coefficients that scale the variance based on the control inputs. Remembering the uncertainty array $\alpha = [\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5, \alpha_6]$, where:

- α_1 : noise proportional to linear velocity v affecting x position.
- α_2 : noise proportional to angular velocity ω affecting x position.

- α_3 : noise proportional to linear velocity v affecting y position.
- α_4 : noise proportional to angular velocity ω affecting y position.
- α_5 : noise proportional to linear velocity v affecting θ (orientation).
- α_6 : noise proportional to angular velocity ω affecting θ (orientation).

Different configurations are included to isolate noise behaviours:

- a: **Ideal case:** $[0, 0, 0, 0, 0, 0]$. No noise, all samples collapse to a single point.
- b: **Pure rotational noise:** $[0, 0, 0.2, 0.2, 0, 0]$. Uncertainty affects ω (angular velocity).
 - **Effect:** the particle cloud forms an arc-shaped (“banana”) distribution, where the robot position is uncertain along a circular trajectory.
- c: **Pure translational noise:** $[0.2, 0.2, 0, 0, 0, 0]$. Uncertainty affects v (linear velocity).
 - **Effect:** particles spread out along the direction of motion.
- d: **Balanced noise:** $[0.05, 0.05, 0.05, 0.05, 0.03, 0.03]$, representing a more realistic setting.

The resulting distributions are visualized in Figure 1.

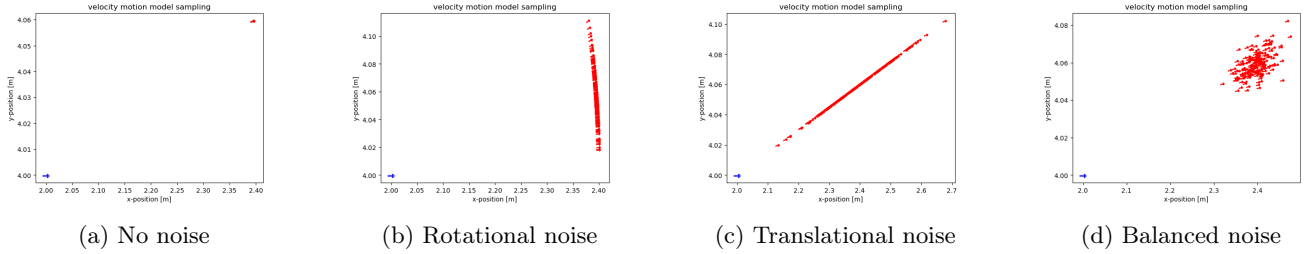


Figure 1: Probabilistic velocity motion model: particle distributions under different noise configurations

Sampling from the Velocity Motion Model As already said, a set of $n_samples$ robot poses was sampled from the velocity-based motion model to empirically verify the predicted state distribution. Each sample represents a realization of the state vector (x, y, θ) after applying the same control input under motion uncertainty. An excerpt of the sampled poses is shown below:

$$\begin{bmatrix} 2.3886 & 4.0587 & 0.30 \\ 2.4033 & 4.0610 & 0.30 \\ 2.4339 & 4.0656 & 0.30 \\ \vdots & \vdots & \vdots \\ 2.3601 & 4.0544 & 0.30 \\ 2.3366 & 4.0509 & 0.30 \\ 2.4196 & 4.0634 & 0.30 \end{bmatrix}.$$

The samples exhibit variability in the planar position (x, y) , while the orientation remains constant, indicating a predominantly translational uncertainty under the selected noise configuration.

Jacobian of the Motion Model The Jacobian of the velocity motion model with respect to the state, denoted by $G_t = \partial g / \partial x$, is computed symbolically as

$$G_t = \begin{bmatrix} 1 & 0 & -\frac{v \cos(\theta)}{\omega} + \frac{v \cos(\theta + \omega \Delta t)}{\omega} \\ 0 & 1 & -\frac{v \sin(\theta)}{\omega} + \frac{v \sin(\theta + \omega \Delta t)}{\omega} \\ 0 & 0 & 1 \end{bmatrix}.$$

Similarly, the Jacobian with respect to the control input $u = [v, \omega]^T$, denoted by $V_t = \partial g / \partial u$, is given by

$$V_t = \begin{bmatrix} -\frac{\sin(\theta)}{\omega} + \frac{\sin(\theta + \omega\Delta t)}{\omega} & \frac{\Delta t v \cos(\theta + \omega\Delta t)}{\omega} + \frac{v \sin(\theta)}{\omega^2} - \frac{v \sin(\theta + \omega\Delta t)}{\omega^2} \\ \frac{\cos(\theta)}{\omega} - \frac{\cos(\theta + \omega\Delta t)}{\omega} & \frac{\Delta t v \sin(\theta + \omega\Delta t)}{\omega} - \frac{v \cos(\theta)}{\omega^2} + \frac{v \cos(\theta + \omega\Delta t)}{\omega^2} \\ 0 & \Delta t \end{bmatrix}.$$

Numerical Evaluation of Jacobians The symbolic Jacobians are evaluated at the state

$$x = [2, 4, 0]^T,$$

and control input

$$u = [v, \omega]^T = [0.8, 0.6]^T,$$

with a time step $\Delta t = 0.5$ s.

The resulting numerical state Jacobian is

$$G_t = \begin{bmatrix} 1 & 0 & -0.0596 \\ 0 & 1 & 0.3940 \\ 0 & 0 & 1 \end{bmatrix},$$

while the evaluated control Jacobian is

$$V_t = \begin{bmatrix} 0.4925 & -0.0198 \\ 0.0744 & 0.0978 \\ 0 & 0.5 \end{bmatrix}.$$

These matrices confirm the expected structure of the velocity motion model: the state Jacobian preserves identity terms for position and orientation, while off-diagonal elements capture the coupling between orientation uncertainty and positional evolution. The control Jacobian reflects how uncertainty in linear and angular velocities propagates differently into position and orientation over the integration interval.

Task 0b: Probabilistic Measurement Model

This task validates the correction step by simulating landmark detections and computing the associated analytical Jacobians.

Analytical Jacobian computation Unlike Task 0a, this script manually computes the Jacobian H_t for the range-bearing model by explicitly deriving the partial derivatives for range and bearing.

Sensor simulation

- **Filtering logic:** The code iterates through a list of landmarks and simulates a sensor with a limited field of view ($\text{fov} = \pi/3$) and maximum range ($\text{max_range} = 6.0$).
- **Probabilistic evaluation:** For each detection, it computes the probability density $p(z \mid x, m)$ of the measurement given the known map and robot state.

Noise parameters analysis Measurement noise is defined by the vector σ . The code uses a medium case of uncertainty:

$$\sigma = [0.3, \pi/24].$$

- $\sigma_1 = 0.3$ (range noise): standard deviation of the distance measurement in meters. Higher values spread the likelihood radially away from the landmark.
- $\sigma_2 = \pi/24 \approx 7.5^\circ$ (bearing noise): standard deviation of the bearing measurement. Higher values spread the likelihood tangentially (along the arc) around the landmark.

The figure below shows the measurement likelihoods for three noise settings.

These parameters populate the measurement covariance matrix Q_t (diagonal matrix of σ^2), determining how much the EKF trusts sensors versus the motion model. Lower σ values result in sharper probability peaks, indicating higher confidence.

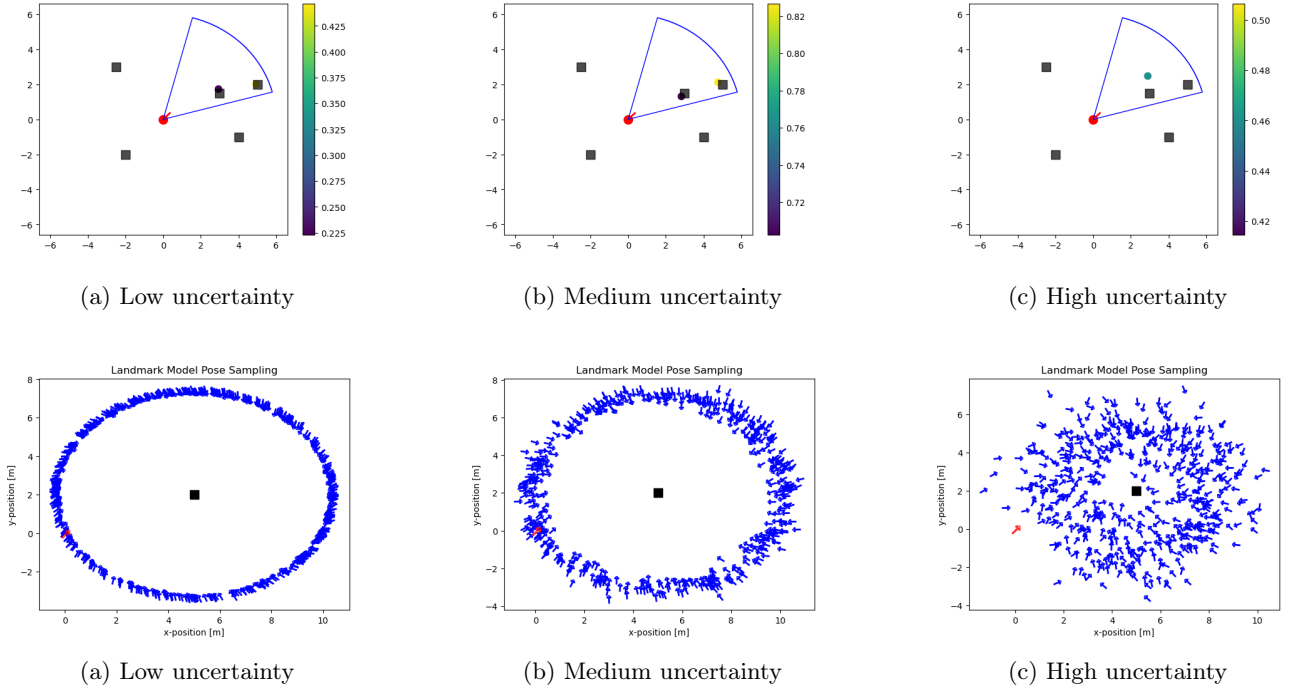


Figure 3: Probabilistic landmark measurement model: measurement likelihoods under different noise configurations

Landmark Measurement Model Verification The probability density values obtained for the simulated landmark measurement are

$$p = [0.8267, 0.7025].$$

The robot pose used for sampling is defined as

$$x = 0.00 \text{ m}, \quad y = 0.00 \text{ m}, \quad \theta = 45.00^\circ,$$

while the landmark is located at

$$m_x = 5.00 \text{ m}, \quad m_y = 2.00 \text{ m}.$$

The simulated sensor measurement consists of a range and bearing given by

$$r = 5.20 \text{ m}, \quad \phi = -14.24^\circ.$$

Jacobian of the Measurement Model The Jacobian matrix $H_t = \partial h / \partial x$ of the range-bearing measurement model, evaluated at the given robot pose and landmark position, is

$$H_t = \begin{bmatrix} \frac{\partial r}{\partial x} & \frac{\partial r}{\partial y} & \frac{\partial r}{\partial \theta} \\ \frac{\partial \phi}{\partial x} & \frac{\partial \phi}{\partial y} & \frac{\partial \phi}{\partial \theta} \end{bmatrix} = \begin{bmatrix} -0.9285 & -0.3714 & 0.0000 \\ 0.0690 & -0.1724 & -1.0000 \end{bmatrix}.$$

The range measurement depends only on the relative position between the robot and the landmark, while the bearing measurement additionally depends on the robot orientation, resulting in a constant derivative of -1 with respect to θ .

Task 1: EKF Localization

Task 1 implements a ROS 2 node (**Task_1**) for tracking the 3-degree-of-freedom (3-DOF) state of the robot. The architecture separates the ROS communication layer from the mathematical logic, using a generic **RobotEKF** class to manage the filter state.

- **State definition:** the filter tracks position (x, y) and orientation (θ) in the global frame.

- **Modular design:** the `Task_1` node initializes `RobotEKF` by injecting specific motion and measurement models (callbacks) rather than hard-coding them. This keeps the core filter logic in `ekf.py` generic and reusable.

Prediction step (time update)

The prediction step propagates the robot state forward in time based on its kinematics.

- **Execution strategy:** the prediction runs in a dedicated timer callback (`prediction_callback`) at a fixed rate of 20 Hz. This ensures the state evolves smoothly even if sensor data is intermittent.
- **Control input u_t :** the node subscribes to `/odom` and treats it as a control input (rather than a measurement). It extracts linear and angular velocity from the twist message to form $u_t = [v, \omega]^T$. This is preferred over using `/cmd_vel` because it reflects executed motion, not commanded motion.

Correction step (measurement update)

The correction step fuses landmark observations to correct drift accumulated during prediction.

- **Sequential updates:** updates are performed inside `landmark_callback`. For each received `LandmarkArray`, the node iterates through detected landmarks and applies the EKF update sequentially. The posterior after landmark i becomes the prior for landmark $i + 1$.
- **Known data association:** the detected ID is matched against a pre-loaded dictionary to retrieve ground truth global coordinates (m_x, m_y) . Unknown IDs are discarded.
- **Measurement model $h(x)$:** the observation model is the range-bearing function.
- **Jacobian computation (H_t):** the Jacobian is computed dynamically for each landmark.

Task 2: Sensor Fusion with Extended State

Task 2 extends the localization architecture to a 5-degree-of-freedom (5-DOF) state in order to fuse multiple sensor modalities. While it retains the modular `RobotEKF` structure and node design from Task 1, the estimation changes from “velocity as control” to “velocity as state”.

- **Extended state vector μ :** the filter estimates $\mu = [x, y, \theta, v, \omega]^T$, explicitly tracking linear and angular velocities alongside pose.
- **Sensor fusion:** this enables the EKF to weigh velocity information (encoders vs IMU) using their covariance matrices rather than blindly trusting odometry as a control input.

Prediction step (constant velocity model)

The prediction logic differs from Task 1. Instead of using an external control input to drive the motion, the robot is modeled with a constant velocity assumption.

Process noise injection (V_t) Uncertainty is injected directly into the velocity states via a 5×2 Jacobian V_t . This maps the process noise variances (`process_noise_v`, `process_noise_omega`) into the velocity components of the state covariance Σ , effectively modeling acceleration noise.

Correction step (multi-sensor fusion)

Task 2 implements three update loops, fusing asynchronous data from landmarks, odometry, and the IMU.

Encoder fusion (velocity correction)

- **Source:** `/odom` twist messages.
- **Measurement model:** The encoder is treated as a direct observation of the velocity states:

$$h_{\text{enc}}(\mu) = [v, \omega]^T.$$

- **Jacobian:** a linear 2×5 matrix selecting indices 3 and 4 (the v and ω components).

- **Effect:** the EKF constrains the internal velocity estimate to track wheel encoders, according to the noise in Q_{enc} .

IMU fusion (angular rate correction)

- **Source:** `/imu` angular velocity (z-axis).
- **Measurement model:**

$$h_{\text{imu}}(\mu) = [\omega].$$

- **Jacobian:** a 1×5 matrix selecting index 4 (ω).
- **Effect:** provides a redundant, typically lower-drift rotational estimate. If wheel slip occurs (encoders indicate rotation but IMU does not), the EKF can arbitrate between sensors through their covariances.

Landmark fusion (pose correction)

- **Source:** `/landmarks` range-bearing observations.
- **Jacobian:** the standard 2×3 Jacobian from Task 1 is expanded to 2×5 :
 - Columns 0–2 contain the spatial derivatives with respect to (x, y, θ) .
 - Columns 3–4 are zero, since landmark measurements provide no direct information about instantaneous velocity.
- **Effect:** landmarks directly correct pose, and cross-correlations in the 5×5 covariance Σ can indirectly correct velocity estimates (e.g., position residuals implying velocity bias).

Task 3: Real Robot Deployment

Task 3 is designed as a deployment-ready wrapper for real robot operation. Unlike Task 1 (3-DOF localization in simulation) and Task 2 (5-DOF sensor fusion), `task_3.py` provides a unified entry point for real hardware.

- **Design goal:** the node does not introduce new mathematical models. Instead, it dynamically instantiates either the Task 1 or Task 2 logic at runtime based on user selection.
- **Code reuse:** it reuses the core `RobotEKF` engine, the timer-based prediction mechanism, and sequential landmark updates from previous tasks.

Dynamic configuration

- **Mode selection:** the main function implements an input loop before spinning the ROS node. This enables switching strategies without recompiling or editing code.
- **Mode 1 (pose-only):** instantiates a 3-DOF EKF (`dim_x=3`), uses odometry as control input, and applies the standard velocity motion model.
- **Mode 2 (pose+velocity):** instantiates a 5-DOF EKF (`dim_x=5`), initializes velocity to zero, uses a constant velocity prediction model, and treats odometry as a measurement.

Prediction callback The prediction callback checks the mode:

- If Mode 1, it uses the last odometry twist as control input u and logs pose-only statistics.
- If Mode 2, it logs the extended state including estimated velocities (v, ω) and uses a zero-control vector, allowing velocity states to propagate internally.

Landmark correction Inside `landmark_callback`, the measurement model $h(\cdot)$ and Jacobian $H(\cdot)$ are selected based on the state dimension:

- **3D branch.** Uses `compute_jacobians` (Task 0b) returning a 2×3 matrix.
- **5D branch.** Uses a local `H_landmark` function returning a 2×5 matrix, padding velocity columns with zeros.

Experiment

Several tests have been performed.

Starting from the task_0 we evaluated the different configuration of noise distributions to then move to task_1 and task_2 in order to perform the algorithm in simulations. Regarding these last two tasks we performed multiple tuning on the noise parameters.

Finally, we deployed the algorithm on the real TurtleBot3 robot to evaluate its performance in a real-world scenario (task_3).

The maps used for the experiments are shown in Figure 4a

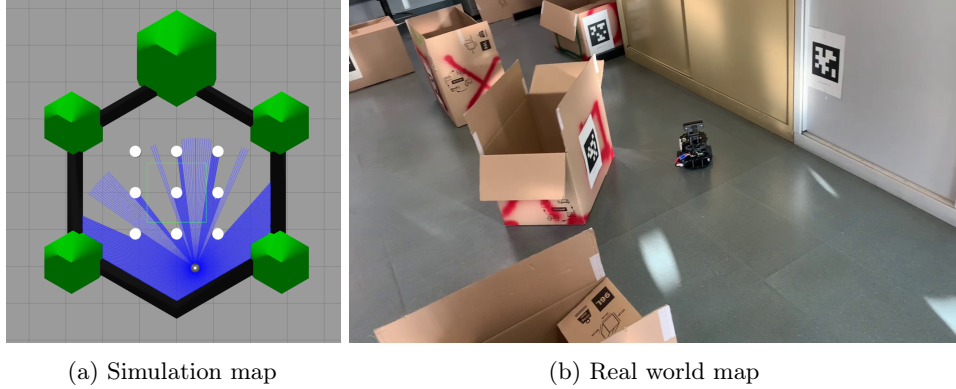


Figure 4: Maps used for the experiments

Results and Discussion

The primary objective of the lab was to implement an Extended Kalman Filter (EKF) for robot localization and to evaluate its behaviour both in simulation and on the real TurtleBot3. In the following results, the trajectories estimated from odometry only are shown in green, while the EKF-corrected trajectories are shown in red.

Task 1: Simulation Experiments

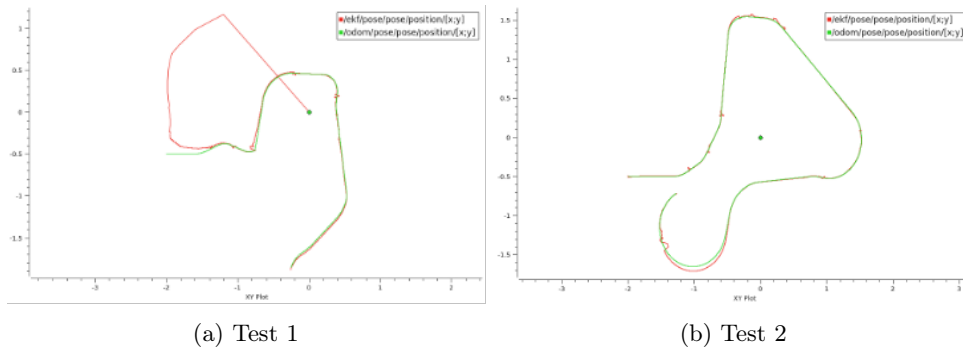


Figure 5: Test of Task1 in simulation

Figure 5a shows the trajectory obtained during the first simulation test. The robot spawn point was fixed by the simulation; however, the EKF initial state was not adjusted accordingly. As a consequence, a large initial offset is visible between the odometry trajectory and the EKF estimate. Despite this initialization mismatch, the two trajectories converge after the first corrections and remain close for the remainder of the run.

In the second simulation test (Figure 5b), the EKF initial pose was set consistently with the robot spawn pose. Under correct initialization, the behaviour of the filter becomes easier to interpret: the two trajectories are almost identical, with a noticeable deviation in the lower part of the map. This discrepancy occurs in a region with few or no landmarks. During the turn, the robot does not detect landmarks, hence no measurement updates are performed

and the EKF relies only on the prediction step, allowing drift to accumulate. Once the robot re-enters an area where landmarks are detected again, measurement updates resume and the EKF estimate converges back toward the odometry trajectory, remaining consistent until the end of the experiment.

Task 3: Real Robot Experiments

Pose-only EKF

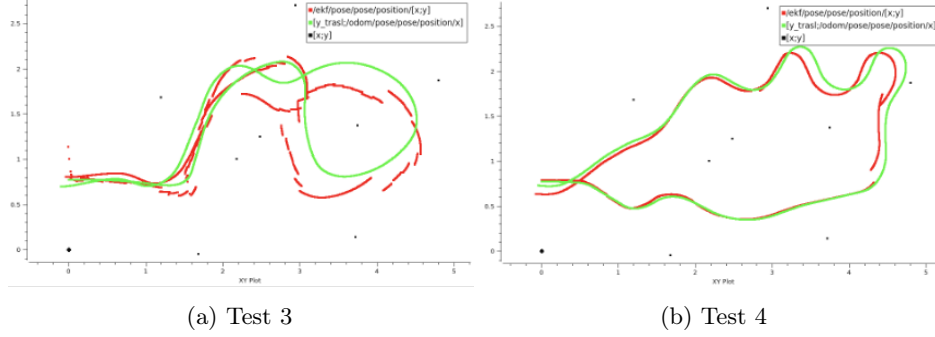


Figure 6: Test of Task3 in real world scenario without sensor fusion

In the real robot tests, the map visualization also includes the landmark locations, shown as black dots. We first evaluated the pose-only EKF configuration (i.e., without velocity-state sensor fusion).

Test 1: Initial parameter tuning

The first real-world experiment (Figure 6a) showed poor performance, mainly due to incorrect noise parameter tuning. The initial values were:

```
process_noise_v      = 0.3
process_noise_omega  = 0.1
measurement_noise_range = 0.1
measurement_noise_bearing = 0.05
```

The TurtleBot3 Burger has a maximum linear speed of approximately 0.22 m/s. Setting `process_noise_v` to 0.3 corresponds to a standard deviation of 0.3 m/s on the linear velocity, which is larger than the maximum achievable speed and is therefore physically unrealistic. With this configuration, the motion model is treated as extremely unreliable, and the EKF tends to overreact to landmark updates. This behaviour is visible as discontinuities (“jumps”) in the estimated trajectory, caused by overly aggressive corrections based on noisy landmark measurements.

Test 2: Improved parameter tuning

To obtain a more stable estimate, the process noise parameters were reduced:

```
process_noise_v      = 0.1
process_noise_omega  = 0.05
measurement_noise_range = 0.1
measurement_noise_bearing = 0.05
```

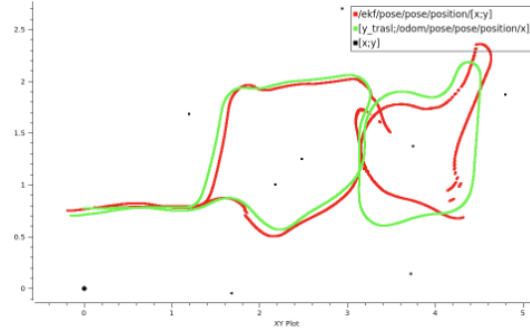
With these values (Figure 6b), the odometry and EKF trajectories become more consistent. Remaining discrepancies are mainly observed on the right side of the map, where the robot detects few landmarks (or only one landmark at a time). In such conditions, the EKF receives limited corrective information and relies more heavily on prediction, leading to temporary divergence.

Interpretation without ground truth Unlike simulation, real experiments do not provide ground-truth robot poses. Therefore, trajectory similarity between odometry and EKF is not, by itself, a proof of correctness. However, in this specific setup the robot moves slowly and over relatively short durations, which typically limits odometric drift. For this reason, an EKF estimate that deviates significantly from odometry is often indicative of poor tuning or an inconsistent measurement model. Conversely, small deviations can reasonably be interpreted as corrective

adjustments that compensate for incremental drift or local measurement effects, especially when they correlate with landmark visibility.

Sensor fusion (extended-state EKF)

When testing the extended-state EKF with sensor fusion, the localization performance degraded compared to the pose-only configuration. The results are shown in Figure 7a.



(a) Test of EKF with sensor fusion

Figure 7: Test of Task3 in real world scenario with sensor fusion

This behaviour suggests that the additional velocity-state updates (encoder and IMU fusion) were not yet tuned consistently with the process and measurement noise assumptions, causing the EKF to incorporate conflicting information in a way that increased estimation error.

Conclusion

In this report, we presented the design, implementation, and evaluation of an Extended Kalman Filter (EKF) for robot localization using ROS 2. The EKF successfully fused odometry and landmark measurements to improve pose estimates in simulation. However, real-world experiments highlighted challenges in parameter tuning and sensor fusion, indicating areas for further refinement.

Additionally, the current implementation does not account for the vertical (z) coordinate of the landmarks, so variations in landmark height are not explicitly considered. This limitation introduces additional uncertainty in the range measurements, particularly when landmarks are positioned at different elevations relative to the robot's sensors.